Web Camera Server Based on Raspberry Pi 4

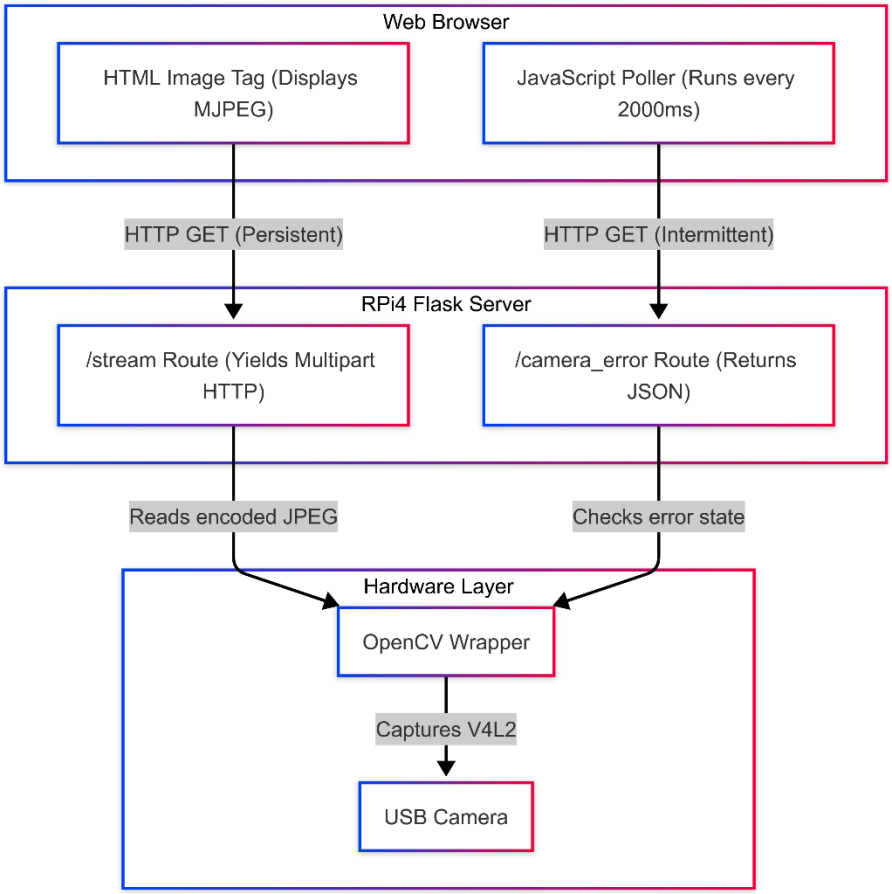
1. System Design Options

To deliver real-time video stream and hardware status updates from a Raspberry Pi 4 to a web browser, two primary system architectures were evaluated.

Option A: MJPEG HTTP Streaming with AJAX Polling (the chosen architecture)

Description:

This architecture relies on standard HTTP protocols. The video stream is served via a continuous, multipart HTTP response (Motion JPEG). The web browser naturally interprets this stream using a standard HTML <img> tag, requiring no client-side decoding logic. To handle hardware status updates (e.g., camera disconnects), the client relies on an asynchronous JavaScript fetch() interval (AJAX polling) that queries a separate JSON endpoint every 2 seconds.

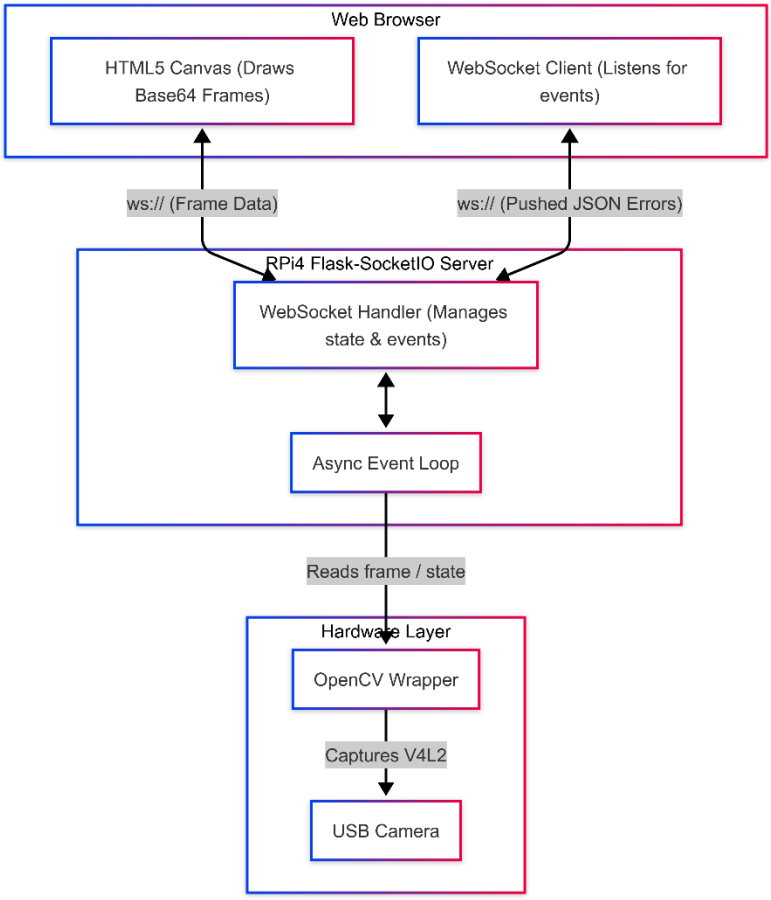


Option A block diagram

Option B: WebSocket Bi-directional Streaming

Description:

This alternative architecture utilizes a single, persistent full-duplex WebSocket connection between the client and the server. Instead of yielding an HTTP stream, the Python backend captures frames, encodes them to Base64 strings, and emits them as WebSocket messages. The client receives these strings and actively draws them to an HTML5 <canvas> element. Hardware status updates and errors are actively "pushed" to the client over the same socket connection the moment they occur, eliminating the need for polling.



Option B block diagram

2. Options Evaluation

The options were evaluated against four critical metrics relevant to a resource-constrained embedded system (Raspberry Pi 4), operating on a local network.

Evaluation Metric	Option A: MJPEG + Polling	Option B: WebSockets
1. Implementation Complexity	**Low:** Utilizes built-in browser features (<img> tag). Python implementation requires only standard Flask routes and a generator function.	**High:** Requires additional dependencies (Flask-SocketIO, eventlets). Requires complex client-side JavaScript to decode Base64 and paint to a canvas at 30 FPS.
2. System Resource Usage (RPi4)	**Low:** Low memory footprint. Encoding to JPEG is highly optimized in OpenCV. Serving multipart HTTP has minimal CPU overhead.	**High:** Base64 encoding frames adds ~33% payload bloat and requires significant CPU cycles on the Pi for every single frame before transmission.
3. Latency & Responsiveness	**Acceptable:** MJPEG provides near real-time video with minimal buffering. Error reporting has up to a 2-second delay due to the polling interval.	**Excellent:** WebSocket pushes allow for instant error notification (0ms delay). Video frame latency can be slightly lower if Base64 overhead doesn't bottleneck the CPU.
4. Network Bandwidth	**Moderate:** MJPEG sends full frames, which is bandwidth-heavy compared to H.264, but manageable on a local Wi-Fi network.	**High:** Base64 encoding increases the sheer size of the data being transmitted over the network by roughly 33%.

3. Option Choice and Justification

Final Decision: Option A (MJPEG HTTP Streaming with AJAX Polling)

<u>Justification:</u>

Option A was selected as the final system design primarily due to its superior alignment with the project's constraints regarding Implementation Complexity and System Resource Usage.

While Option B (WebSockets) is technical superior, instantaneous error reporting and a modernized full-duplex protocol, it introduces severe CPU overhead by forcing the Raspberry Pi to Base64-encode video frames at 30 FPS. This overhead risks dropping frames and causing thermal throttling on the Pi.

Option A offloads the rendering complexity entirely to the browser's native <img> tag parsing. Furthermore, for a localized hardware monitor, a 2.0-second delay in error reporting (via AJAX polling) is an acceptable trade-off to maintain low CPU overhead, ensuring the server remains stable, responsive, and easy to maintain.